48-Hour Hackathon Guidelines

GENESIS
INNOVATE, IDEATE, INSPIRE

Event Overview

This event is an intensive, continuous 48-hour hackathon designed to push the boundaries of creativity and technical problem-solving. Participants will collaborate in a high-energy environment to conceptualize, design, and build functional software or hardware prototypes.

Team Requirements

Requirement	Details
Team Size	2 members
Registration Note	All team members must be registered individually prior to the event kickoff. Solo participation is not permitted to ensure a collaborative environment.

General Guidelines

1. Development & Code Originality

- Fresh Code Only: All project development—including coding, designing, and asset creation—must take place entirely within the 48-hour hackathon window.
- No Pre-built Projects: You may not work on pre-existing projects or submit work that was started before the official hacking period begins.
- Open Source & Boilerplates: The use of open-source libraries, frameworks, and publicly available boilerplates is highly encouraged, provided they are freely accessible to all participants. You must clearly declare any third-party assets used in your final submission.

2. Submission Requirements

- Deadline Strictness: The submission portal will lock exactly at the 48-hour mark. Late submissions will not be accepted under any circumstances.

- Required Deliverables:
  - A link to a public source code repository (e.g., GitHub, GitLab).
  - A short demonstration video (maximum 3 minutes) showcasing the working features of the prototype.
  - A brief report following the specified format below.

Project Report Format

The report must be included in your submission and follow this structure:

1. The Problem Statement

- The Challenge: A brief description (3–4 sentences) of the specific real-world problem or pain point the project aims to solve.
- Target Audience: Who is the primary user or beneficiary of this solution?

2. The Solution & Core Features

- Value Proposition: A clear explanation of how the project directly addresses the stated problem.
- Completed Features: A bulleted list of the functional features successfully built during the 48-hour window.
- Incomplete Features: A brief note on planned features that were not finished (this demonstrates strong scope awareness and honesty).

3. Technical Architecture & Tech Stack

- Frontend: Languages, frameworks, and libraries used for the user interface.

- Backend & Database: Server infrastructure, database systems, and hosting platforms.
- APIs & Third-Party Tools: Any external APIs, cloud services, open-source libraries, or datasets integrated into the project.

- System Architecture: A short text description outlining how data flows between the frontend, backend, and external services.

Software & Application Policies

To maintain a level playing field and focus on raw technical skill, please adhere to the following software policies.

Apps Allowed

Participants are encouraged to use standard industry tools (examples shown) to facilitate their development and design process.

Category	Examples / Permitted Tools
Integrated Development Environments (IDEs)	VS Code, IntelliJ, Eclipse, Xcode, Android Studio, WebStorm
Version Control & Collaboration	GitHub, GitLab, Bitbucket, Slack, Discord, Microsoft Teams, Notion, Trello
Design & Prototyping	Figma, Adobe XD, Sketch, Canva, Miro
Cloud & Deployment	AWS, Google Cloud Platform (GCP), Microsoft Azure, Vercel, Heroku, Netlify, Firebase
APIs & Testing	Postman, Insomnia, Swagger, and any publicly available API (e.g., Google Maps API, Stripe API)
AI Assistants (as tools)	GitHub Copilot, ChatGPT, or Claude (for debugging, code suggestions, and brainstorming)

Apps Not Allowed

Category	Restrictions
"No-Code" App Builders	Wix, Bubble, Glide, Webflow, or similar platforms where the entire application is generated without writing custom code (unless specifically approved for a non-technical category).
Proprietary / Closed-Source Internal Tools	Any enterprise or private software that is not publicly available to all other teams, giving an unfair advantage.
Fully Autonomous AI Agents	Tools like Devin or AutoGPT used to autonomously generate and deploy the entire codebase without human intervention.
Malicious Software	DDoS tools, network sniffers (like Wireshark, unless explicitly used on your own local traffic for debugging), penetration testing suites (like Metasploit), or any app used to disrupt the event network.